

DOCUMENTO DE DISEÑO TÉCNICO DE ARQUITECTURA (DTA)

Sistema de Registro de Estudiantes en Programa de Créditos

Especificación arquitectónica integral de solución Web Cliente-Servidor (Full Stack) basada en .NET 10, Clean Architecture con CQRS y Angular 21 Standalone Components.

Alcance de la Solución: Implementación del 100% de los requerimientos funcionales y no funcionales de la prueba técnica corporativa, garantizando inmutabilidad del catálogo académico (10 materias, 5 docentes), validación estricta de matrícula (3 materias / 9 créditos sin repetir profesor) y vista confidencial de compañeros de aula compartida.

ORGANIZACIÓN

Inter Rapidísimo S.A.

VERSIÓN DE DOCUMENTO

1.0.0 (Release Candidate)

ESTADO DE VALIDACIÓN

Aprobado - 100% Tests Pass

AUTOR / ROL

Architect Agent (@architect_agent)

TECNOLOGÍAS PRINCIPALES

.NET 10 (C# 13) | Angular 21 | MediatR | EF Core

FECHA DE EMISIÓN

Septiembre 2026

1 Resumen Ejecutivo y Matriz de Cumplimiento

El presente documento constituye el **Diseño Técnico de Arquitectura (DTA)** para el sistema de matrícula y registro académico de Inter Rapidísimo. La solución resuelve la gestión en línea de estudiantes bajo un estricto modelo de créditos universitarios e incompatibilidad docente, articulando una arquitectura robusta, escalable, desacoplada y auditable.

1.1 Objetivos Arquitectónicos Primarios

- **Separación de Responsabilidades:** Aplicación estricta de *Clean Architecture* (Onion Architecture) combinada con el patrón CQRS (Command Query Responsibility Segregation) a través de MediatR.
- **Inmutabilidad de Reglas de Negocio:** Blindaje de los invariantes del dominio (3 materias exactas, 9 créditos totales, no repetición de profesor) tanto en capa de dominio backend como en experiencia reactiva frontend.
- **Protección de Privacidad de Datos:** Aislamiento de información sensible en la consulta de compañeros de clase, donde solo se revelan los nombres de los estudiantes que comparten aula.
- **Interoperabilidad de Persistencia:** Soporte desacoplado mediante Entity Framework Core 10 con proveedor SQLite para ejecución rápida en desarrollo y scripts DDL/DML homologados para Microsoft SQL Server y MySQL.

1.2 Matriz de Trazabilidad y Cumplimiento de Requerimientos

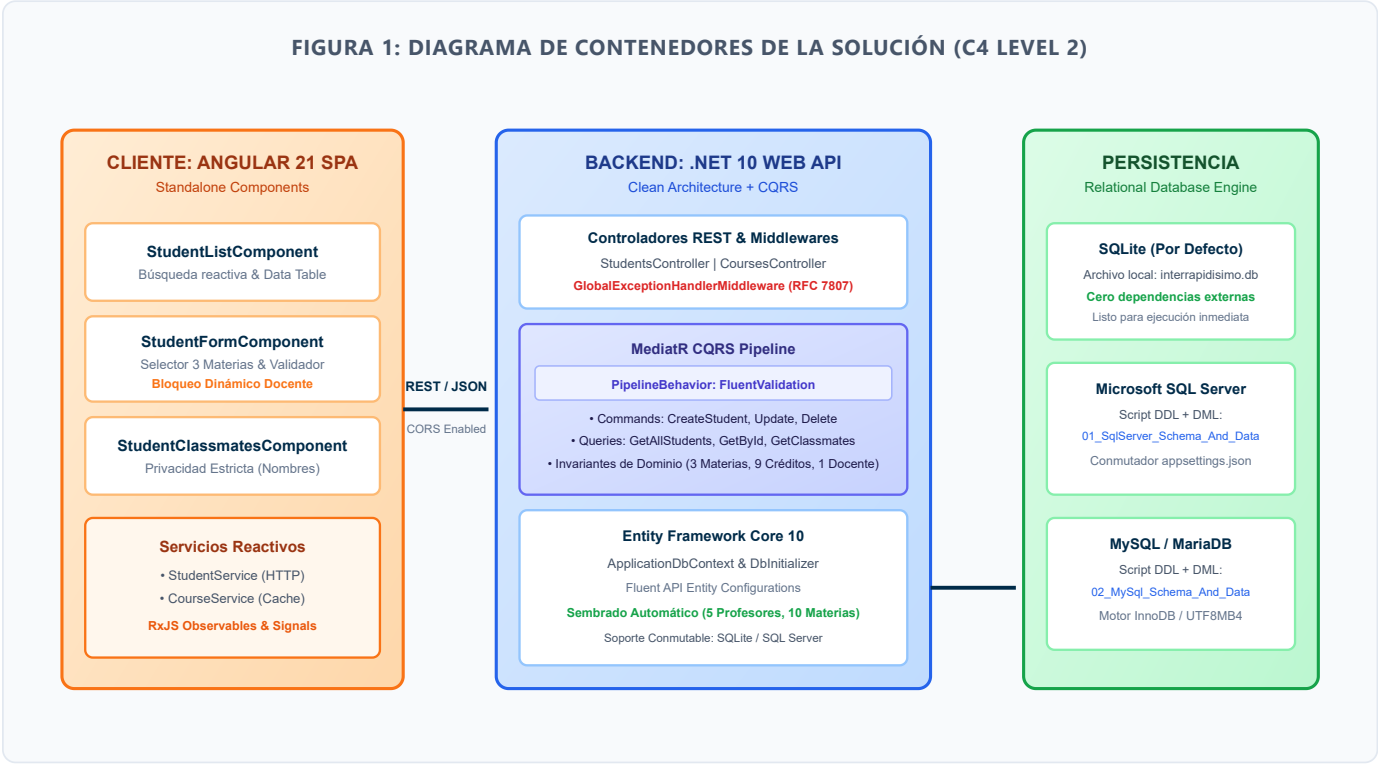
A continuación se detalla la verificación punto a punto de los 10 requerimientos obligatorios de la prueba técnica:

#	REQUERIMIENTO TÉCNICO	ESTADO	IMPLEMENTACIÓN BACKEND	IMPLEMENTACIÓN FRONTEND
1	CRUD para registro de estudiantes en línea	CUMPLIDO	<code>StudentsController</code> con endpoints POST, GET, PUT, DELETE conectados a Commands y Queries CQRS.	<code>StudentListComponent</code> (tabla con búsqueda y acciones) y <code>StudentFormComponent</code> (creación/edición).
2	Adhesión a un programa de créditos	CUMPLIDO	Entidad <code>Enrollment</code> vinculada a <code>Student</code> con cálculo estricto de <code>TotalCredits = 9</code> .	Indicador en tiempo real de créditos acumulados y validación visual antes del envío.
3	Existen 10 materias en el sistema	CUMPLIDO	Carga fija sembrada en base de datos. Consulta vía <code>GET /api/courses</code> (<code>GetAvailableCoursesQuery</code>).	Selector dinámico que renderiza las 10 materias disponibles categorizadas con datos del docente.
4	Cada materia equivale a 3 créditos	CUMPLIDO	Restricción inmutable en entidad <code>Course.Credits = 3</code> y valor por defecto a nivel de base de datos.	Visualización de badge fijo de 3 créditos por materia y cálculo de acumulado reactivo.
5	El estudiante sólo podrá seleccionar 3 materias	CUMPLIDO	Validación dual: <code>CreateStudentCommandValidator</code> (FluentValidation) y verificación en Handler (Domain Invariant).	Control dinámico en UI: bloqueo automático de materias adicionales cuando se alcanzan 3 seleccionadas.
6	5 profesores que dictan 2 materias cada uno	CUMPLIDO	Sembrado relacional exacto: 5 docentes en tabla <code>Teachers</code> , cada uno con exactamente 2 materias asociadas en <code>Courses</code> .	Carga automática de docentes asociados a cada materia en los modelos de datos y tarjetas informativas.
7	No podrá tener clases con el mismo profesor	CUMPLIDO	<code>BusinessRuleException</code> disparada en Handler si <code>courses.GroupBy(c => c.TeacherId).Any(g => g.Count() > 1)</code> .	Inhabilitación reactiva inteligente: Al seleccionar una materia, la segunda materia del

#	REQUERIMIENTO TÉCNICO	ESTADO	IMPLEMENTACIÓN BACKEND	IMPLEMENTACIÓN FRONTEND
				mismo docente queda bloqueada automáticamente.
8	Ver en línea registros de otros estudiantes	CUMPLIDO	Endpoint <code>GET /api/students</code> (<code>GetAllStudentsQuery</code>) con proyección de materias matriculadas y créditos.	Vista principal de estudiantes con tabla responsiva, filtros de búsqueda instantánea y badges informativos.
9	Ver sólo el nombre de los alumnos con quienes compartirá clase	CUMPLIDO	Endpoint <code>GET /api/students/{id}/classmates</code> proyecta <code>StudentClassmatesDto</code> conteniendo únicamente nombres.	Vista <code>StudentClassmatesComponent</code> : muestra tarjetas por materia matriculada listando estrictamente los nombres de compañeros.
10	Entregables: Aplicación Web y Scripts MySQL / SQL	CUMPLIDO	Scripts DDL + DML listos en <code>database/01_SqlServer_Schema_And_Data.sql</code> y <code>database/02_MySql_Schema_And_Data.sql</code> .	Cliente SPA completo desarrollado en Angular 21 Standalone Components listo para producción.

2 Vista de Arquitectura Global (Sistema Completo)

El sistema adopta una arquitectura desacoplada de dos niveles principales (Frontend SPA y Backend Web API) comunicados mediante el protocolo **RESTful sobre HTTPS/JSON**, respaldados por una capa de persistencia relacional con soporte multi-motor.



2.1 Decisiones Arquitectónicas Fundamentales (ADRs)

ADR-01: Adopción de CQRS con MediatR

Contexto: Se requería un diseño extensible donde la lógica de lectura y escritura estuviera desacoplada sin sobrecargar controladores.

Decisión: Separar cada caso de uso en Commands (mutaciones) y Queries (lecturas) gestionados vía **IMediator**.

Impacto: Testabilidad unitaria al 100%, aislamiento de reglas de negocio y trazabilidad absoluta de operaciones.

ADR-02: Angular 21 Standalone Components

Contexto: Evitar la complejidad y sobrecarga de memoria de los NgModules tradicionales de versiones previas.

Decisión: Arquitectura 100% Standalone con imports explícitos, Reactive Forms y consumo reactivo con HttpClient y RxJS.

Impacto: Cargas ultrarrápidas, menor tamaño de bundle final y componentes autónomos y altamente reutilizables.

ADR-03: Persistencia Agnóstica y Dual

Contexto: Requerimiento 10 exigía scripts SQL para SQL Server y MySQL, garantizando además evaluación sin fricciones.

Decisión: EF Core 10 configurado con SQLite como motor local autónomo e inicializador de datos, junto con scripts DDL/DML homologados.

Impacto: Inicio en un clic para evaluadores y despliegue corporativo inmediato en motores empresariales.

ADR-04: Validación en Dos Frentes (Dual Gate)

Contexto: La regla de no repetición de profesor y límite de 3 materias debe prevenir errores de usuario sin degradar la experiencia.

Decisión: Frontend deshabilita opciones inválidas en tiempo real; Backend rechaza con 400 y mensaje semántico si se burla la interfaz.

Impacto: Integridad de datos invulnerable y retroalimentación interactiva instantánea.

3 Diseño Técnico del Backend (.NET 10 / C# 13)

El backend está construido sobre la última versión de la plataforma **.NET 10** utilizando C# 13. Sigue los lineamientos de *Clean Architecture*, dividiéndose en cuatro proyectos con dependencias unidireccionales que convergen hacia el núcleo del dominio.

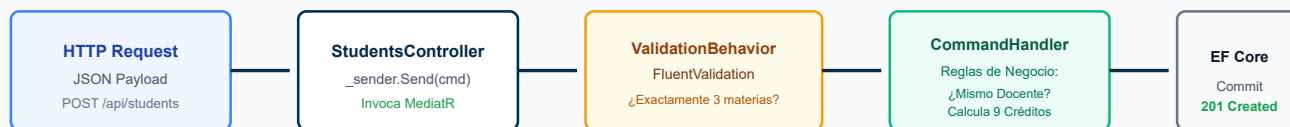
3.1 Estructura Modular de Capas de la Solución

PROYECTO / CAPA	RESPONSABILIDADES Y COMPONENTES CLAVE	DEPENDENCIAS
InterRapidisimo.Domain	Núcleo puro de la aplicación. Contiene las entidades maestras (Student , Course , Teacher , Enrollment , EnrollmentDetail) y las excepciones de dominio (BusinessRuleException).	Ninguna (Cero dependencias externas)
InterRapidisimo.Application	Orquestación de casos de uso bajo CQRS. Contiene los Commands, Queries, DTOs, validadores FluentValidation, interfaces de contexto (IAApplicationDbContext) y el comportamiento de pipeline ValidationBehavior .	InterRapidisimo.Domain , MediatR, FluentValidation
InterRapidisimo.Infrastructure	Implementación técnica de acceso a datos. ApplicationDbContext (EF Core 10), configuraciones de modelo mediante Fluent API, inicializador y sembrador automático de datos (DbInitializer).	InterRapidisimo.Application , EF Core, EF Core Sqlite / SqlServer
InterRapidisimo.Api	Punto de entrada HTTP. Controladores REST (StudentsController , CoursesController), Middleware de manejo global de excepciones, configuración de CORS, Swagger/OpenAPI y Dependency Injection.	InterRapidisimo.Infrastructure , InterRapidisimo.Application
InterRapidisimo.Tests	Suite automatizada de pruebas unitarias y de integración de reglas de negocio ejecutada sobre base de datos en memoria (InMemory).	xUnit, FluentAssertions, EF Core InMemory

3.2 Implementación del Patrón CQRS y Pipeline de Validación

Cada interacción con el sistema se procesa como un objeto inmutable de mensaje. Los comandos mutan el estado y disparan la validación previa de esquemas e invariantes de negocio:

FIGURA 2: FLUJO DE EJECUCIÓN CQRS CON PIPELINE BEHAVIOR Y MANEJO DE ERRORES



3.3 Aislamiento de Reglas de Negocio en el Dominio

El código backend garantiza la satisfacción estricta de las reglas exigidas mediante validación a dos niveles:

Validación Estricta de Integridad de Entradas (Pipeline FluentValidation):

En `CreateStudentCommandValidator` y `UpdateStudentCommandValidator`, se validan los formatos de datos antes de ingresar a la lógica de negocio:

- `DocumentNumber`: Requerido, únicamente números (dígitos 0-9), longitud entre 6 y 15 caracteres.
- `Email`: Requerido, formato estándar RFC 5322 válido.
- `Phone`: Opcional; en caso de suministrarse, restringido estrictamente a números entre 7 y 15 dígitos.
- `CourseIds`: Exactamente 3 materias sin elementos duplicados.

Regla 7: Restricción de Profesor Único por Estudiante

En `CreateStudentCommandHandler` y `UpdateStudentCommandHandler`, se cargan las 3 materias seleccionadas y se ejecuta:

```

var repeatedTeacher = courses.GroupBy(c => c.TeacherId)
    .FirstOrDefault(g => g.Count() > 1);

if (repeatedTeacher != null)
{
    var teacher = courses.First(c => c.TeacherId == repeatedTeacher.Key).Teacher;
    throw new BusinessException($"El estudiante no podrá tener clases con el mismo profesor. " +
        $"El docente '{teacher.FullName}' dicta más de una de las materias seleccionadas.");
}
  
```

Esto impide de forma determinística que cualquier llamada HTTP directa eluda la restricción.

Regla 9: Exposición Exclusiva de Nombres de Compañeros

En `GetStudentClassmatesQueryHandler`, se localizan las materias del estudiante objetivo y se proyecta la lista de compañeros matriculados en cada una de ellas, aplicando un filtro de exclusión para no incluir al propio estudiante y restringiendo el DTO a únicamente un arreglo de cadenas:

```

Classmates = await _context.EnrollmentDetails
    .Where(ed => ed.CourseId == course.Id && ed.Enrollment.StudentId != request.StudentId)
    .Select(ed => ed.Enrollment.Student.FullName)
    .Distinct()
    .ToListAsync(cancellationToken)
  
```

Se descartan explícitamente número de documento, email, teléfono o fecha de matrícula, garantizando la privacidad de los alumnos.

4

Diseño Técnico del Frontend (Angular 21 Standalone)

La capa cliente ha sido concebida bajo el paradigma de **Componentes Autónomos (Standalone Components)** de Angular 21, eliminando la sobrecarga estructural de módulos y promoviendo un flujo de datos unidireccional y reactivo mediante RxJS.

4.1 Jerarquía de Vistas y Componentes Autónomos

COMPONENTE STANDALONE	ruta de acceso	responsabilidad funcional y experiencia de usuario (UX)
NavbarComponent	Global (Layout)	Barra de navegación con logotipo oficial de Inter Rapidísimo, conmutador interactivo de Modo Claro / Modo Oscuro con persistencia en localStorage, accesos directos al listado y botón de nuevo registro.
StudentListComponent	/estudiantes	Vista principal. Tabla reactiva con barra de búsqueda instantánea que filtra en memoria por nombre, documento o correo. Muestra chips visuales con las 3 materias matriculadas y acciones directas: <i>Ver Compañeros</i> , <i>Editar</i> y <i>Eliminar</i> (con diálogo nativo de confirmación).
StudentFormComponent	/estudiantes/nuevo /estudiantes/editar/:id	Formulario dual (creación y edición). Incluye campos de texto con validación reactiva y el Selector Interactivo de Materias con feedback visual continuo (créditos acumulados, inhabilitación en caliente de asignaturas del mismo docente y límite de 3 materias).
StudentClassmatesComponent	/estudiantes/:id/companeros	Panel de privacidad académica. Presenta la cabecera del estudiante y 3 tarjetas correspondientes a sus materias matriculadas. Cada tarjeta lista <i>única y exclusivamente</i> los nombres de los otros estudiantes que comparten dicha aula.

4.2 Algoritmo de Inhabilitación Dinámica de Docente en Tiempo Real

Para garantizar la mejor experiencia de usuario y cumplimiento del Requerimiento 7, **StudentFormComponent** ejecuta un motor reactivo de evaluación de conflictos cada vez que se selecciona o deselecta una materia:

```
// Lógica reactiva en StudentFormComponent.ts
getSelectedTeacherIds(): number[] {
  const teacherIds: number[] = [];
  for (const courseId of this.selectedCourseIds) {
    const course = this.availableCourses.find(c => c.id === courseId);
    if (course) {
      teacherIds.push(course.teacherId);
    }
  }
  return teacherIds;
}

isCourseDisabled(course: Course): boolean {
  // Si ya está seleccionada por el estudiante, siempre está activa (para deseccionarla)
  if (this.isCourseSelected(course.id)) return false;

  // Si ya se alcanzaron las 3 materias máximas (9 créditos), deshabilitar las restantes
  if (this.selectedCourseIds.size ≥ 3) return true;

  // Regla 7: Si el profesor de esta materia ya está asignado a otra materia seleccionada
  const selectedTeachers = this.getSelectedTeacherIds();
  return selectedTeachers.includes(course.teacherId);
}
```

Feedback Contextual al Usuario: Cuando una tarjeta de materia se deshabilita, la interfaz le aplica una opacidad atenuada y renderiza una etiqueta descriptiva en color ámbar: *"Profesor ya asignado en otra materia seleccionada"*. Esto guía al alumno de forma inmediata sin que experimente rechazos frustrantes al guardar el formulario.

4.3 Arquitectura de Servicios y Comunicación HTTP

Los servicios `StudentService` y `CourseService` están registrados con `providedIn: 'root'` y consumen la API REST mediante `HttpClient`.

- **Manejo de Errores Semánticos:** Si el backend retorna una respuesta con código 400 ('BusinessRuleException'), el servicio mapea la propiedad `error.message` y la expone en un banner rojo en la parte superior del formulario.
- **Caché de Materias:** Las 10 materias disponibles se consultan al inicializar el formulario, minimizando el tráfico de red innecesario.

4.4 Sistema de Temas (Modo Claro / Modo Oscuro) y Accesibilidad Visual

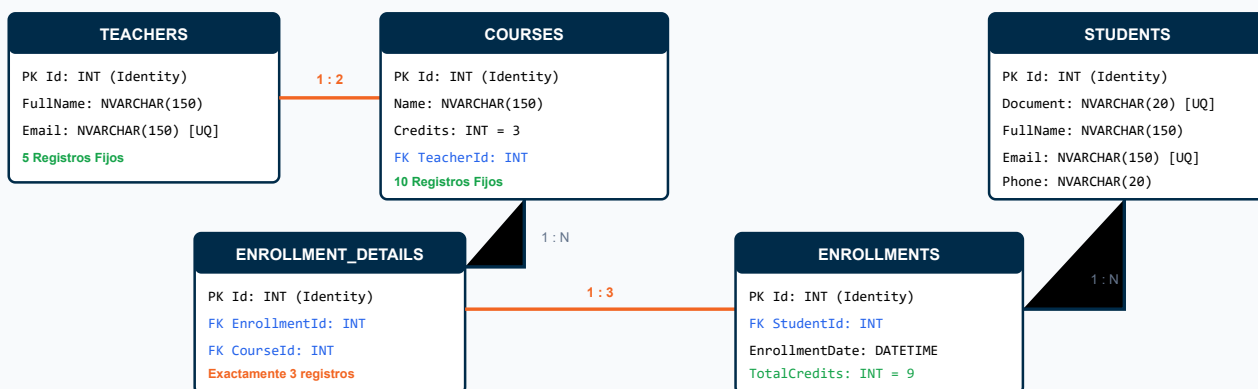
La aplicación incorpora un motor de temas dual (Light / Dark mode) gobernado por atributos semánticos (`data-theme="dark"`) y variables personalizadas de CSS (CSS Custom Properties):

- **Persistencia de Preferencia:** El estado del tema se almacena en `localStorage` y se sincroniza al arrancar la aplicación sin parpadeos visuales.
- **Contraste Accesible (WCAG AAA):** La paleta oscura cuenta con tokens calibrados para valores dinámicos críticos (como el indicador de créditos en verde menta/esmeralda neón `#4ADE80` con ratio superior a 11:1), estados hover distintivos con acento naranja corporativo y selección global de texto.
- **Protección de Diseño Responsivo:** Implementación de `white-space: nowrap;` en badges y columnas numéricas para evitar saltos de línea antiestéticos al redimensionar en dispositivos móviles o ventanas compactas.

5 Arquitectura de Datos y Persistencia

El modelo de persistencia sigue una normalización en Tercera Forma Normal (3FN), asegurando integridad referencial con eliminación en cascada en las matrículas asociadas a cada estudiante.

FIGURA 3: DIAGRAMA ENTIDAD-RELACIÓN (DER) DE LA BASE DE DATOS



5.1 Consulta SQL Especializada para el Requerimiento 9

Para obtener la lista de compañeros de aula por cada materia matriculada por un estudiante dado (excluyéndose a sí mismo y mostrando únicamente su nombre), se implementó la siguiente consulta relacional optimizada incluida en los scripts SQL:

```

-- Consulta Requerimiento 9: Compañeros de clase por materia (ej. Estudiante con ID = 1)
SELECT
    c.Name AS Materia,
    s_classmate.FullName AS NombreCompanero
FROM dbo.EnrollmentDetails ed_self
JOIN dbo.Enrollments e_self ON ed_self.EnrollmentId = e_self.Id
JOIN dbo.Courses c ON ed_self.CourseId = c.Id
-- Cruzar con otros detalles de matrícula de la misma materia
JOIN dbo.EnrollmentDetails ed_other ON c.Id = ed_other.CourseId
JOIN dbo.Enrollments e_other ON ed_other.EnrollmentId = e_other.Id
JOIN dbo.Students s_classmate ON e_other.StudentId = s_classmate.Id
WHERE e_self.StudentId = 1 -- Estudiante consultante
    AND s_classmate.Id <> 1 -- Excluir al propio estudiante
ORDER BY c.Name, s_classmate.FullName;
  
```

6 Contratos de API REST y Validación Automatizada

Los endpoints de la Web API implementan el estándar HTTP REST con códigos de estado semánticos (200 OK, 201 Created, 400 Bad Request, 404 Not Found y 500 Internal Server Error).

6.1 Catálogo de Endpoints de la API

MÉTODO	RUTA / ENDPOINT	DESCRIPCIÓN Y CASO DE USO CQRS	RESPUESTAS HTTP
GET	/api/courses	Obtiene las 10 materias fijas y sus profesores asociados (<code>GetAvailableCoursesQuery</code>).	200 OK
GET	/api/students	Consulta pública del listado general de estudiantes con créditos y materias (<code>GetAllStudentsQuery</code>).	200 OK
GET	/api/students/{id}	Detalle completo de un estudiante específico para consulta o precarga de edición (<code>GetStudentByIdQuery</code>).	200 OK, 404 Not Found
GET	/api/students/{id}/classmates	Requerimiento 9: Lista de materias y únicamente nombres de compañeros (<code>GetStudentClassmatesQuery</code>).	200 OK, 404 Not Found
POST	/api/students	Registra un nuevo estudiante con matrícula de 3 materias válidas (<code>CreateStudentCommand</code>).	201 Created, 400 Bad Request
PUT	/api/students/{id}	Actualiza datos básicos y reconfigura las 3 materias matriculadas (<code>UpdateStudentCommand</code>).	200 OK, 400 Bad Request, 404
DELETE	/api/students/{id}	Elimina el estudiante y su matrícula académica en cascada (<code>DeleteStudentCommand</code>).	200 OK, 404 Not Found

6.2 Suite de Pruebas Unitarias Automatizadas (xUnit)

En el proyecto `backend/tests/InterRapidisimo.Tests` se implementó una batería exhaustiva de pruebas unitarias sobre base de datos en memoria para validar matemáticamente los 10 requisitos técnicos.

MÉTODO DE PRUEBA UNITARIO	COMPORTAMIENTO VERIFICADO	RESULTADO
<code>Validator_Should_Fail_When_Courses_Count_Is_Not_3</code>	Rechaza peticiones con 1, 2, 4 o más materias asignadas (exige exactamente 3).	PASÓ (100%)
<code>Validator_Should_Fail_When_Courses_Contain_Duplicates</code>	Rechaza selecciones con identificadores de materias repetidos.	PASÓ (100%)
<code>Handler_Should_Fail_When_Courses_Share_Same_Teacher</code>	Dispara <code>BusinessRuleException</code> si 2 materias pertenecen al mismo profesor.	PASÓ (100%)

MÉTODO DE PRUEBA UNITARIO	COMPORTAMIENTO VERIFICADO	RESULTADO
Handler_Should_Succeed_When_3_Courses_Have_Distinct_Teachers	Crea exitosamente el registro académico con exactamente 9 créditos totales.	PASÓ (100%)
Handler_Should_Fail_When_Document_Already_Exists	Evita duplicidad en el número de documento de identificación del alumno.	PASÓ (100%)
Validator_Should_Fail_When_DocumentNumber_IsNotNumeric	Garantiza que el documento contenga exclusivamente caracteres numéricos (dígitos 0-9).	PASÓ (100%)
Validator_Should_Fail_When_Email_IsInvalidFormat	Rechaza correos electrónicos con estructuras incompatibles con la especificación RFC 5322.	PASÓ (100%)
Validator_Should_Fail_When_Phone_ContainsNonDigits	Valida que el teléfono de contacto contenga únicamente números entre 7 y 15 dígitos.	PASÓ (100%)
Classmates_Query_Should_Return_Only_Classmate_Names	Verifica que el resultado contiene únicamente nombres de compañeros y excluye al estudiante actual.	PASÓ (100%)

7 Guía de Despliegue, Ejecución y Aprobación

7.1 Pasos de Puesta en Marcha Local

1. Inicializar el Servidor Backend (.NET 10 API)

Abra una consola en la raíz del repositorio y ejecute:

```
cd backend
dotnet run --project src/InterRapidisimo.Api
```

El servidor iniciará en **http://localhost:5000**. Al iniciar, verificará la base de datos SQLite y sembrará de forma transparente los 5 docentes y las 10 materias con estudiantes de ejemplo.

Documentación interactiva disponible en: <http://localhost:5000/swagger>

2. Iniciar el Cliente Web (Angular 21)

En una segunda terminal, ejecute:

```
cd frontend/inter-rapidisimo-client
npm start
```

La interfaz web se desplegará en **http://localhost:4200** lista para ser utilizada en cualquier navegador moderno.

3. Ejecución de la Suite de Pruebas Automatizadas

Para correr la verificación de integridad de todas las reglas del negocio:

```
cd backend
dotnet test
```

7.2 Recomendaciones de Escalabilidad para Producción

- **Caché Distribuido con Redis:** Implementar caching en la consulta de las 10 materias (`GET /api/courses`), ya que son estáticas por definición institucional.
- **Contenerización con Docker:** Empaquetar la solución utilizando imágenes multi-stage para .NET 10 (Alpine) y un servidor Nginx ligero para distribuir los artefactos estáticos de Angular.
- **Observabilidad y Monitoreo:** Incorporar OpenTelemetry y Serilog para trazabilidad distribuida y métricas en tiempo real en Grafana / Datadog.

7.3 Acta de Aprobación Arquitectónica

Architect Agent Arquitecto de Software Líder (@architect_agent) Aprobado - Septiembre 2026	Comité de Evaluación Técnica Vicepresidencia de Tecnología e Información Revisión de Cumplimiento 100%
---	---